MeddyNet Full Repo
Deep Audit Report

Repository: arpit0381/meddynet_full Date: May 28, 2026
Scope: All 5 sub-projects — Backend API + 4 Next.js frontends

7
Critical crash-level bugs

5
Security vulnerabilities

9
Missing backend modules

200+
Lint warnings (patient app)

Overall Production Readiness

CONFIDENTIAL — MeddyNet Production Readiness Audit 2026

Table of Contents

1. Executive Summary

This report presents an exhaustive forensic audit of the MeddyNet monorepo (arpit0381/meddynet_full), a multi-tenant diagnostic lab aggregation platform consisting of five sub-projects: a FastAPI Python backend, and four Next.js 16 frontends serving patients, labs, technicians, and administrators.

The core booking-payment-lab-report loop is architecturally sound and works end-to-end. However, the platform contains seven crash-level runtime bugs, five exploitable security vulnerabilities, nine completely unbuilt backend modules, nine admin portal pages still powered by static mock data arrays, and essentially no automated test coverage. The platform is NOT currently safe for live production deployment without addressing at minimum the P0 items in the action plan.

Metric	Value	Severity
Critical runtime bugs	7	P0 — fix before deploy
Security vulnerabilities	5	P0 — fix before deploy
Missing backend modules	9	P1 — build next sprint
Admin pages on mock data	9 / ~20	P1 — connect to live API
Lint warnings (patient app)	200+	P2 — clean up
Total backend test coverage	~5%	P2 — write tests
Frontend test coverage	0%	P2 — add Playwright/Jest
CI/CD pipeline	None	P3 — set up GitHub Actions

2. Repository Structure Overview

The repository is a monorepo containing one Python FastAPI backend and four Next.js 16 TypeScript frontends, plus a GitHub Actions configuration directory and several documentation files. The primary language split is TypeScript (87.8%) and Python (11.5%).

Path	Description	Status
`meddynet-backend/`	FastAPI + SQLAlchemy + NeonDB + MongoDB	Core — has bugs
`MeddyNet/`	Patient-facing Next.js 16 app (PWA)	~75% complete
`admin.meddynet/`	Admin dashboard — Next.js 16	~55% — heavy mock data
`meddynet-lab-portal/`	Lab partner portal — Next.js 16	~80% complete
`technician-app/`	Technician PWA — Next.js 16	~70% complete
`.github/workflows/`	GitHub Actions CI/CD	EMPTY — no pipelines
`MeddyNet_Pending_Features.md`	9 unbuilt PRD features documented	Reference doc
`meddynet_deep_analysis.md`	Existing analysis notes	Should move to /docs/
`MeddyNet_Executive_Summary.md`	Previous audit summary	Should move to /docs/
`Today_Work_Report.md`	Dev work log	Clutter — delete or archive
`backend_doc.txt / sys_arch.txt`	Loose documentation files	Move to /docs/
`*.docx files (2)`	System architecture Word docs	Move to /docs/

3. Critical Bugs & Runtime Issues

Seven bugs were identified that will cause crashes, data loss, or silent failures in production. All must be resolved before any live deployment.

#3.1 [CRASH] Unreachable code in audit middleware

`meddynet-backend/app/main.py — lines 58–74`

The exception handler calls raise e which re-throws the exception immediately. Any code below that line — including the duration calculation, slow-request logging (lines 60–73), and the return response statement — is completely unreachable whenever an error occurs. This means the performance monitoring system is effectively disabled on all error paths.

```
except Exception as e:
logger.error(f'Unhandled exception on {request.url.path}', exc_info=True)
raise e # <-- re-raises here

duration = time.time() - start_time # <-- DEAD CODE on error path
if duration > 2.0:
logger.warning(...) # <-- also DEAD CODE
return response # <-- also DEAD CODE
```

Fix: Wrap the duration calculation and logging in a finally: block so it executes regardless of success or failure.

#3.2 [CRASH] WebSocket imports non-existent async_session

`meddynet-backend/app/websocket.py`

The WebSocket handler attempts to import async_session from app.database. However, the database module only exports get_db and SessionLocal — async_session does not exist anywhere in the codebase. Any WebSocket event of type support_message will trigger an ImportError crash, making the entire live patient-to-admin chat feature completely non-functional at runtime.

```
from app.database import async_session # <-- DOES NOT EXIST

# app/database.py exports:
# get_db (dependency injection function)
# SessionLocal (synchronous session factory)
# NOT async_session
```

Fix: Either create an async_session factory in database.py, or refactor the WebSocket handler to use get_db() correctly with an async context manager.

#3.3 [BUG] Double db.commit() in booking creation

`meddynet-backend/app/routers/bookings.py — lines 139 and 185`

The booking creation handler calls await db.commit() twice: once correctly after the database write on line 139, and again unnecessarily on line 185 after building the response dictionary. While typically a no-op, the second commit adds latency and under concurrent booking load could create race conditions with other pending transactions.

```
await db.commit() # line 139 -- correct
logger.info(...)
# ... build booking_data response dict ...
await db.commit() # line 185 -- REDUNDANT
```

Fix: Remove the second db.commit() on line 185. No other changes needed.

#3.4 [BUG] Razorpay key secret stripping bug

`meddynet-backend/app/services/payment_service.py`

The code strips the 'live_' or 'test_' prefix from Razorpay API keys using str.replace(), which replaces ALL occurrences of the substring anywhere in the string — not just at the prefix position. For a hypothetical key like 'live_secret_live_xyz', the result would be 'secretxyz' (corrupted), causing payment API authentication to fail silently.

```
if self.key_secret.startswith('live_'):
self.key_secret = self.key_secret.replace('live_', '') # BUG
# Should be: self.key_secret = self.key_secret.removeprefix('live_')
```

Fix: Replace all uses of .replace('live_', '') and .replace('test_', '') with .removeprefix('live_') and .removeprefix('test_') respectively. Python 3.9+ built-in, already compatible with your runtime.

#3.5 [BUG] Webhook HMAC crashes on missing Razorpay signature header

`meddynet-backend/app/routers/webhooks.py — line 37`

The webhook endpoint declares x_razorpay_signature as an Optional[str] header defaulting to None. However, hmac.compare_digest() on line 37 is called without first checking whether the value is None. Any webhook request that arrives without this header (network error, misconfiguration, or an attacker probing the endpoint) will crash with a TypeError instead of returning a clean 400 Bad Request response.

```
x_razorpay_signature: Optional[str] = Header(None)
# ...
hmac.compare_digest(expected_sig, x_razorpay_signature)
# TypeError: compare_digest expects str, got NoneType
```

Fix: Add an early guard: if not x_razorpay_signature: raise HTTPException(status_code=400, detail='Missing signature header')

#3.6 [SILENT FAILURE] MongoDB DNS failure silently kills all analytics

`meddynet-backend — startup log output`

At application startup, MongoDB Atlas fails to connect with the error: 'The DNS query name does not exist' for cluster cluster0.6lzhtrl.mongodb.net. The cluster is either deleted, renamed, or on a free tier that was paused due to inactivity. The consequence is that all audit logging, notification history, admin audit log endpoint, and the analytics dashboard silently return empty data — with no error surfaced to the user.

```
# Startup log:
Failed to initialize MongoDB indexes:
The DNS query name does not exist
# cluster0.6lzhtrl.mongodb.net is unreachable
```

Fix: Create a new MongoDB Atlas cluster (or resume the existing one), update the MONGODB_URL in .env, and verify index creation succeeds at startup.

#3.7 [DEPRECATED] @app.on_event('startup') will break in future FastAPI

`meddynet-backend/app/main.py`

FastAPI deprecated the @app.on_event('startup') and @app.on_event('shutdown') decorators in version 0.93 in favour of the lifespan context manager pattern. The current code generates deprecation warnings and will break in a future FastAPI major version upgrade.

```
@app.on_event('startup') # DEPRECATED
async def startup():
await init_db()
```

```
# Should be:
from contextlib import asynccontextmanager
@asynccontextmanager
async def lifespan(app: FastAPI):
await init_db()
yield
app = FastAPI(lifespan=lifespan)
```

Fix: Migrate to the lifespan context manager pattern. FastAPI docs have a direct migration guide.

4. Security Vulnerabilities

Five security issues were identified ranging from credential exposure to authentication bypass risks. Two of these (secrets in Git and missing login rate limiting) are high-severity vulnerabilities that must be resolved immediately.

#4.1 [CRITICAL — SECRETS] .env file contains live production credentials

`meddynet-backend/.env`

The .env file in the workspace contains real, live credentials including: NeonDB PostgreSQL connection strings (with username and password), Supabase anon and service role keys, Razorpay test/live API keys, Cloudinary cloud name and API secrets, Authkey SMS API key, JWT secret and Fernet encryption keys. If this file was EVER committed to Git history (even once, then removed), ALL of these credentials are accessible to anyone with read access to the repository and must be rotated immediately. Check with: git log --all --full-history -- .env

Fix: IMMEDIATE ACTION: Run git log --all -- .env. If any result appears, the file was committed. Rotate every credential across all services today. Then add .env to .gitignore and use .env.example with placeholder values.

#4.2 [CRITICAL — AUTH] No rate limiting on /auth/login endpoint

`meddynet-backend/app/routers/auth.py`

The OTP send endpoint (/auth/send-otp) correctly has the @limiter.limit('5/minute') decorator. However, the password-based /auth/login endpoint has absolutely no rate limiting decorator applied. This means an attacker can attempt unlimited password guesses per second with no throttling, making brute-force attacks trivially easy against any patient or lab account.

```
# /auth/send-otp — CORRECTLY protected:
@limiter.limit('5/minute')
async def send_otp(...): ...

# /auth/login — NO PROTECTION:
async def login(...): # <-- missing @limiter.limit()
...
```

Fix: Add @limiter.limit('10/minute') above the login function. Also consider adding account lockout after 5 failed attempts.

#4.3 [HIGH — 2FA BYPASS] OTP code leaked in HTTP development response

`meddynet-backend/app/routers/auth.py`

When ENVIRONMENT is set to 'development', the OTP code is returned directly in the HTTP response body as dev_otp. If someone accidentally deploys the application with ENVIRONMENT=development in production (a common mistake), every 2FA OTP code is transmitted over the wire in plaintext, completely bypassing the SMS verification flow.

```
if settings.ENVIRONMENT == 'development':
response['dev_otp'] = otp # <-- OTP in HTTP response
```

Fix: Remove this block entirely. For development debugging, log the OTP to the server console with logger.debug() — it never needs to be in the HTTP response.

#4.4 [HIGH — RBAC] superadmin role blocked from admin endpoints

`meddynet-backend/app/routers/admin_portal.py`

The check_admin dependency function only grants access when role equals the string 'admin'. However, the rbac.py module also defines and uses a 'superadmin' role with elevated permissions. A user with role='superadmin' attempting to access any admin endpoint will receive a 403 Forbidden error, making the superadmin role non-functional for administrative tasks.

```
def check_admin(current_user):
if current_user.role != 'admin': # blocks superadmin
raise HTTPException(403)

# rbac.py defines: require_role('admin'), require_role('superadmin')
# These are inconsistent
```

Fix: Change the check to: if current_user.role not in ('admin', 'superadmin'): raise ...

#4.5 [HIGH — DATA INTEGRITY] User deletion has no cascade protection

`meddynet-backend/app/routers/admin_portal.py`

The admin user deletion endpoint performs a hard delete with await db.delete(user) without first checking for related records (bookings, payments, lab reports, technician assignments). In production with real data, this will cause PostgreSQL foreign key constraint violations, resulting in a 500 error and a partially corrupted database state.

```
await db.delete(user) # no cascade check
await db.commit() # may raise ForeignKeyViolation
```

Fix: Implement soft-delete: add an is_deleted boolean column to the User model and set it to True instead of performing a hard delete. Alternatively, check all related tables before deletion and return a 409 Conflict if related records exist.

5. Incomplete & Half-Built Features

5.1 Missing backend modules — no model, no router, no API

Nine features are documented in the PRD and visually present in the admin portal UI, but have zero backend implementation. Each one renders static mock data from a TypeScript array in admin.meddynet/src/data/ rather than fetching from a live API endpoint.

Module	Backend status	Frontend status	PRD priority	Effort
Subscription / Plans engine	**No model, no router**	Admin uses mock data	**CRITICAL**	2–3 days
Coupon / Promo system	**No model, no router**	Admin uses mock data	**CRITICAL**	1 day
Global master test catalog	**No MasterTest model**	Admin uses mock data	**HIGH**	1 day
City / Zone operations	**No model, no router**	Admin uses mock data	**HIGH**	0.5 day
Blog / CMS	**No model, no router**	Admin + patient mock	**MEDIUM**	1 day
Background data export	**No Celery task**	Admin uses mock data	**MEDIUM**	1 day
Reviews API	**Model only, no router**	Admin + patient mock	**MEDIUM**	0.5 day
Firebase push notifications	**Not integrated**	No device token mgmt	**HIGH**	1 day
Patient chat / support tickets	**Admin side only**	Patient UI broken	**HIGH**	1–2 days

5.2 Admin portal — 9 pages still on mock data

The following admin pages read from static TypeScript data files rather than live API endpoints. Any data shown in the admin portal for these pages is entirely fictional and does not reflect real platform state.

Mock data file	Admin page	Backend API status
`coupons.ts`	/admin/coupons	**NO API EXISTS**
`reviews.ts`	/admin/reviews	**No router (model exists)**
`financials.ts`	/admin/financials	**Partially live**
`labs.ts`	/admin/labs	**Partially live**
`users.ts`	/admin/users	**Partially live**
`bookings.ts`	/admin/bookings	**Partially live**
`support.ts`	/admin/support	**Partially live**
`technicians.ts`	/admin/technicians	**Partially live**
`notifications.ts`	/admin/notifications	**Via MongoDB only**

5.3 Hardcoded technician queue ETA

The technician portal dashboard displays a real-time estimated queue time, but the value is hardcoded as the integer 12 with a comment acknowledging it is simulated. This means every technician always sees '12 minutes' regardless of actual workload.

```
"queue_time_mins": 12, # Still simulated based on fleet avg
```

This needs to be replaced with a real calculation based on active job queue length, technician availability, and historical completion times.

5.4 Reports audit returns hardcoded placeholder data

The admin report audit endpoint returns completely fabricated response values for lab name, test name, file size, and report status. An admin viewing this endpoint is seeing no real information about the actual reports on the platform.

```
"lab": "Partner Lab", # hardcoded "test": "Diagnostic Panel", # hardcoded "size": "1.5 MB", #
fake "status": "Clean" # fake
```

6. Code Quality Issues

#6.1 [PERFORMANCE] async_sessionmaker recreated on every HTTP request

`meddynet-backend/app/database.py`

The async_sessionmaker factory — an expensive object that should be created once at application startup — is being instantiated fresh inside the get_db() dependency function, which runs on every single API request. This adds unnecessary overhead to every endpoint call.

```
async def get_db():
async_session = async_sessionmaker(pg_engine, ...) # BAD: every request
```

Fix: Move the async_sessionmaker(...) call to module level. Create it once when the database.py module is first imported.

#6.2 [CRASH] Technician.city column referenced but does not exist in model

`meddynet-backend/app/models/technician.py + routers/admin_portal.py`

The admin portal technician list endpoint references t.city on the Technician ORM object. The Technician SQLAlchemy model has no city column defined. This will crash with an AttributeError the first time the technician list admin endpoint is called.

```
# admin_portal.py:
'city': t.city, # AttributeError — Technician has no .city column
```

Fix: Add city: Mapped[str] = mapped_column(String(100), nullable=True) to the Technician model and run an Alembic migration.

#6.3 [SCALABILITY] No pagination on any list endpoint

`meddynet-backend/app/routers/* — all GET list endpoints`

Every list endpoint across the entire API — including GET /admin/users, GET /admin/bookings, GET /admin/labs, GET /bookings, GET /technicians — returns ALL database records with no limit or offset parameter. With production-scale data (10,000+ bookings, 500+ labs), these endpoints will consume gigabytes of memory, take many seconds to respond, and freeze the frontend.

Fix: Add ?page=1&limit;=20 query parameters to all list endpoints using FastAPI's Query() dependency. Apply .offset(skip).limit(limit) to all SQLAlchemy select statements.

#6.4 [LINT] 200+ lint warnings in patient app

`MeddyNet/ (patient Next.js app)`

The lint_final_readable.txt log file in the repository reveals an extensive list of issues: dozens of unused imports (Filter, Search, Calendar, Sparkles, Info, and many more icon imports), multiple @typescript-eslint/no-explicit-any violations throughout the codebase, and missing return type annotations. These indicate rapid development with copy-paste patterns and insufficient code review.

Fix: Run: npx eslint --fix && npx tsc --noEmit. Remove all unused imports. Replace all 'any' types with proper TypeScript interfaces.

#6.5 [MAINTAINABILITY] Inconsistent import patterns across backend routers

`meddynet-backend/app/routers/*`

Imports are written inconsistently across the backend codebase. Some modules import dependencies at the top of the file (standard Python style), while others import inside function bodies (from app.config import settings inside login()). Imports inside functions execute on every call, can mask circular import bugs, and make the code harder to read and maintain.

Fix: Run isort across the entire backend: isort meddynet-backend/. Move all function-level imports to the top of their respective files.

7. Testing Gaps

The platform has virtually no automated test coverage. The entire backend has only 7 smoke tests covering basic endpoint availability. The four frontend applications have zero tests of any kind. For a healthcare platform handling medical reports, payments, and patient data, this is a critical production risk.

Test type	Current state	Required coverage	Priority
Backend smoke tests	**7 tests exist**	Basic endpoint availability	**Done (expand)**
Backend auth tests	**Partial (test_auth.py)**	OTP, JWT, 2FA, refresh, lockout	**P2**
Booking + payment flow	**MISSING**	Create booking, initiate payment, webhook reconcile, confirm	**P2 — critical**
Report upload flow	**MISSING**	Upload PDF, Cloudinary store, patient access	**P2 — critical**
Lab onboarding flow	**MISSING**	Register lab, admin verify, add tests, go live	**P2**
Technician job assignment	**MISSING**	Assign job, accept, complete, report upload	**P2**
Admin operations	**MISSING**	Verify lab, suspend user, process payout, manage coupons	**P2**
WebSocket events	**MISSING**	Connect, send message, receive notification, disconnect	**P2**
Patient app — unit tests	**ZERO**	Components, hooks, API calls, form validation	**P2**
E2E tests (Playwright)	**ZERO**	Full booking flow, login, report download	**P2**

8. DevOps & Infrastructure

#8.1 [MISSING] No CI/CD pipeline — .github/workflows/ is empty

`.github/workflows/`

The .github/workflows/ directory exists in the repository but contains zero YAML workflow files. There is no automated pipeline for linting, testing, building, or deploying any of the five applications. Every deployment is manual, meaning bugs could reach production undetected.

Fix: Create three workflow files: ci.yml (run eslint + pytest on every PR), build.yml (build all Next.js apps on merge to main), deploy.yml (deploy to Vercel/Railway on release tag).

#8.2 [BROKEN] Docker Compose uses deprecated version key

`docker-compose.yml (repo root)`

The docker-compose.yml file uses version: '3.8' at the top, which is a deprecated attribute in Docker Compose V2 and generates warnings. Additionally, Docker Desktop was not running during the audit, making the entire containerised development environment non-functional.

```
version: '3.8' # DEPRECATED — Compose V2 ignores this attribute
```

Fix: Remove the version key entirely (Compose V2 does not require it). Fix any env file bindings. Test with: docker compose up --build

#8.3 [CLEANUP] Orphan Meddynet_Backend/ folder with single text file

`Meddynet_Backend/ (repo root)`

A directory named Meddynet_Backend/ exists at the repo root containing only a single file named docx2.txt. This is clearly an abandoned artefact from a previous iteration of the project. It has no connection to any other code and serves no purpose.

Fix: Delete the entire Meddynet_Backend/ directory. Commit with message: 'chore: remove orphan backend directory'

#8.4 [CLEANUP] node_modules_old/ directories in 3 frontend apps

`admin.meddynet/ · meddynet-lab-portal/ · technician-app/`

Three of the four Next.js applications contain a node_modules_old/ directory which is the result of renaming node_modules/ during package manager switching. Each of these directories is likely 300–600MB in size and serves no purpose. They should not be tracked by Git (verify in .gitignore).

*Fix: Delete all three: rm -rf admin.meddynet/node_modules_old/ meddynet-lab-portal/node_modules_old/ technician-app/node_modules_old/. Ensure */node_modules_old is in .gitignore.*

#8.5 [CLEANUP] Loose documentation files pollute the repo root

`/ (repo root)`

The repository root contains a mixture of documentation files that should live in a dedicated /docs/ directory: Today_Work_Report.md, backend_doc.txt, sys_arch.txt, meddynet_deep_analysis.md, MeddyNet_Executive_Summary.md, MeddyNet_Pending_Features.md, and two .docx files. The root should only contain README.md, .gitignore, docker-compose.yml, and package.json (if using workspaces).

*Fix: mkdir docs && mv *.md *.txt *.docx docs/ (except README.md). Update any cross-references.*

9. Production Readiness Status by Component

The table below summarises the current completion and health status for each of the five sub-projects and cross-cutting concerns, along with the primary gap preventing each from being production-ready.

Component	Completion	Health	Key gap
Backend API core (Auth, Bookings, Payments, Labs)	~82%	**7 bugs to fix**	P0 crash bugs, no pagination
Patient app (MeddyNet)	~75%	**Mostly functional**	200+ lint issues, some mocks
Lab portal	~80%	**Good shape**	Minor cleanup needed
Technician app	~70%	**Missing real-time**	Hardcoded queue ETA, partial features
Admin dashboard	~55%	**Heavy mock data**	9 pages not connected to live API
Missing backend modules	~0%	**Not built**	Subscriptions, Coupons, CMS, Cities, FCM, Reviews, Export
Backend test coverage	~5%	**Critical gap**	7 smoke tests; core flows untested
Frontend test coverage	0%	**Critical gap**	Zero tests across all 4 Next.js apps
DevOps / CI pipeline	~10%	**Not set up**	No pipeline, broken Docker, no automation

10. Prioritised Action Plan — 32 Items

Items are sorted by urgency. P0 items must be resolved before any production deployment. P1 items are required for full PRD feature parity. P2 items are quality and reliability improvements. P3 items are cleanup and hygiene.

#	Priority	Task	File / area	Effort
1	P0	Rotate ALL secrets if .env was ever committed to Git	`.env`	2 hrs
2	P0	Fix WebSocket async_session import crash	`websocket.py`	10 min
3	P0	Fix unreachable code in audit middleware (finally block)	`main.py`	15 min
4	P0	Add rate limiting to /auth/login	`auth.py`	5 min
5	P0	Fix check_admin RBAC to also allow superadmin	`admin_portal.py`	5 min
6	P0	Fix double db.commit() in booking creation	`bookings.py`	5 min
7	P0	Fix Razorpay replace() → removeprefix()	`payment_service.py`	5 min
8	P0	Add None-header guard to Razorpay webhook HMAC	`webhooks.py`	5 min
9	P0	Add cascade check / soft-delete before user deletion	`admin_portal.py`	30 min
10	P0	Fix MongoDB Atlas cluster DNS / create new cluster	`infra / .env`	30 min
11	P1	Build Subscription/Plans engine (model + router + Razorpay Subs)	`new module`	2–3 days
12	P1	Build Coupon/Promo system + checkout flow integration	`new module`	1 day
13	P1	Build Firebase FCM push notification integration	`new module`	1 day
14	P1	Connect Admin Reports page to live API (write useAdminReports hook)	`hooks.ts`	2 hrs
15	P1	Build City / Zone operations module	`new module`	0.5 day
16	P1	Build Reviews API router (list, moderate, respond)	`reviews.py`	0.5 day
17	P1	Build Global Master Test Catalog (model + CRUD)	`new module`	1 day
18	P1	Build Blog/CMS (model + CRUD + Cloudinary + public endpoints)	`new module`	1 day
19	P1	Build async Background Data Export (Celery + CSV/Excel)	`export_tasks.py`	1 day
20	P2	Add pagination to ALL list endpoints (page + limit params)	`routers/*`	1 day
21	P2	Fix 200+ lint warnings in MeddyNet patient app	`MeddyNet/`	0.5 day
22	P2	Move async_sessionmaker to module-level singleton	`database.py`	15 min
23	P2	Add Technician.city column + Alembic migration	`models/technician.py`	15 min
24	P2	Migrate @app.on_event('startup') to lifespan context manager	`main.py`	30 min
25	P2	Remove OTP from HTTP dev response (log-only instead)	`auth.py`	5 min
26	P2	Write integration tests: booking, payment, auth, webhook	`tests/`	2–3 days

#	Priority	Task	File / area	Effort
27	P2	Write E2E tests for patient app (Playwright)	`MeddyNet/e2e/`	2 days
28	P3	Set up GitHub Actions CI/CD (lint + pytest + build + deploy)	`workflows/`	2 hrs
29	P3	Fix Docker Compose (remove deprecated version key, fix env)	`docker-compose.yml`	30 min
30	P3	Delete orphan Meddynet_Backend/ folder and 3x node_modules_old/	`repo root`	5 min
31	P3	Move loose docs to /docs/ folder, clean repo root	`repo root`	10 min
32	P3	Standardise all imports to file-top (run isort across backend)	`routers/*`	30 min

11. Strategic Recommendations

11.1 Fix P0 bugs in one session (2–3 hours)

All ten P0 items are small, isolated fixes. None requires architectural changes. A focused engineer can resolve all of them in a single afternoon: the five one-liners (rate limit, removeprefix, RBAC fix, double commit, OTP response), then the WebSocket import, unreachable finally block, cascade deletion guard, and MongoDB cluster fix. The secrets rotation may require coordination with Razorpay, Supabase, and Cloudinary dashboards.

11.2 Build Subscriptions + Coupons before any other P1 module

The PRD's monetisation strategy depends entirely on the Subscription engine and Coupon system. These are the features that convert the booking platform from a cost centre into a revenue generator. They should be the first P1 modules built, followed by Firebase FCM (which enables user retention) and the Reviews API (which enables trust signals for lab acquisition).

11.3 Connect admin portal to live APIs in a dedicated sprint

Nine admin pages showing fake data represents a significant operational risk. Administrators making business decisions (verifying labs, processing payouts, reviewing support tickets) based on mock data is dangerous. Dedicate a full sprint to replacing all mock data files with live TanStack Query hooks pointing to existing or newly-built API endpoints.

11.4 Implement a testing baseline before scaling up features

With only 7 tests for the entire backend and zero frontend tests, every new feature deployment is a bet. At minimum, write integration tests covering the three critical user journeys: the full booking-payment-confirmation flow, the report-upload-download flow, and the lab-onboarding-verification flow. These three test suites will catch the majority of regressions as the codebase grows.

11.5 Set up GitHub Actions CI before the next developer joins

Without a CI pipeline, there is no automated gatekeeper on code quality. Adding a second developer to this codebase without CI means both engineers will regularly break each other's work. Set up at minimum a PR check that runs: eslint across all four frontend apps, pytest across the backend, and next build on the patient app and admin portal.

11.6 Add pagination immediately — before data volume grows

Pagination is one of those issues that becomes exponentially harder to fix retroactively as data accumulates. With zero pagination on any endpoint, the first 1,000 real bookings will noticeably slow the admin dashboard. The first 10,000 will make it unusable. This is a relatively mechanical change (add offset/limit to every select query) that should be done before onboarding the first real lab partner.

Report generated: May 28, 2026 | Scope: arpit0381/meddynet_full (15 commits, main branch) | Total issues catalogued: 32 | Critical (P0): 10 | High (P1): 9 | Medium (P2): 8 | Low (P3): 5